

Gerador de números pseudo-aleatórios com detecção de não repetição baseada em Handshaking: Relatório Final de Projeto Fase II

Integrante(s): Matheus Grossi

Data: 26 de março de 2026

Tecnologia Alvo: SkyWater 130nm / FPGA Xilinx Spartan-3E (90nm)

Sumário

1	Introdução e Especificação	1
2	Implementação RTL	3
3	Estratégia de Verificação e Resultados	6
4	Síntese Lógica e Resultados PPA	9
5	Conclusão e Lições Aprendidas	16
6	Referências e Apêndices	17

1. Introdução e Especificação

- **Resumo Executivo:** Este projeto consiste na implementação em Verilog de um Gerador de Números Pseudoaleatórios (PRNG). O sistema foi projetado para gerar números de 3 bits (faixa de 0 a 7) utilizando 4 sementes (seeds) distintas baseadas em tabelas de permutação.

- **Arquitetura do Sistema:**

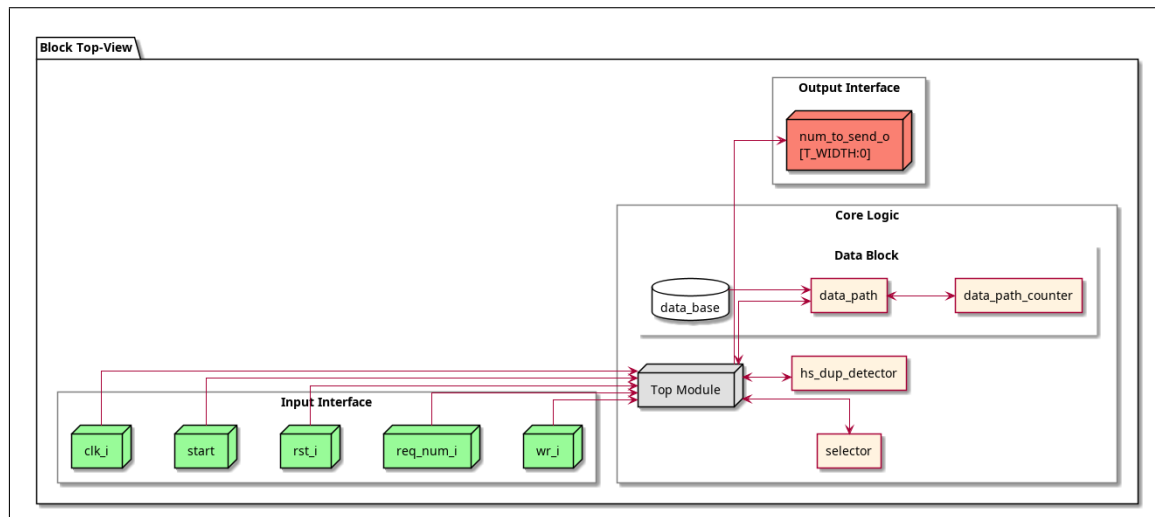


Figura 1: Representação em blocos da arquitetura do sistema

- **Tabela de Interfaces:**

Sinal	Dir.	Bits	Descrição funcional
clk_i	I	1	Clock principal do sistema.
rst_i	I	1	Reset assíncrono ativo em nível baixo.
req_num_i	I	1	Requisição de geração de um novo número pseudoaleatório.
wr_i	I	1	Confirmação de escrita/aceite; ao ser ativado, o valor válido é salvo no histórico.
num_to_send_o	O	3	Saída de dado pseudoaleatório validado pelo detector de duplicatas.

Tabela 1: Interface funcional do módulo de topo rng_top.

- **Conformidade com Protocolos:** Para a detecção de duplicidade há um sistema íterno de handshaking para detectar se os valores que chegam já são existentes no banco de memória e caso sejam são descartados e solicitam uma nova amostra.
- **Mapa de Registradores:** Tabela detalhando endereços, permissões (R/W) e funções de cada bit.

Nome	Tipo.	Bits	Descrição funcional
ram_0	reg.	3	1º Bit do barramento de memória
ram_1	reg.	3	2º Bit do barramento de memória
ram_2	reg.	3	3º Bit do barramento de memória
ram_3	reg.	3	4º Bit do barramento de memória
ram_4	reg.	3	5º Bit do barramento de memória
ram_5	reg.	3	6º Bit do barramento de memória
ram_6	reg.	3	7º Bit do barramento de memória
ram_7	reg.	3	8º Bit do barramento de memória

Tabela 2: Registradores internos do validador de handshaking implementado na bancada UVM.

2. Implementação RTL

A implementação RTL do bloco foi desenvolvida em Verilog com foco em portabilidade entre FPGA e ASIC. O circuito gera valores pseudoaleatórios de **3 bits**, na faixa de 0 a 7, e incorpora um mecanismo de **não repetição** baseado em histórico interno e protocolo de handshaking. A arquitetura foi organizada de forma hierárquica para separar geração de candidatos, seleção de sementes, controle temporal e detecção de duplicatas.

Estratégia de design

O módulo de topo é o `rng_top`, responsável por integrar os sub-blocos funcionais e expor a interface externa do sistema. A hierarquia descrita na documentação-base é composta pelos seguintes blocos:

- `rng_top`: integra controle, caminho de dados e lógica de não repetição;
- `rng_selector`: seleciona uma entre **4 seeds** por meio de um contador *free-running*, produzindo o índice da tabela de permutação ativa;
- `rng_data_path`: multiplexa o valor numérico candidato com base na seed selecionada e na posição corrente da sequência;
- `rng_data_path_counter`: contador cíclico de 3 bits que percorre as posições das tabelas internas;
- `rng_hs_dup_detector`: armazena o histórico dos valores já aceitos, compara o candidato atual com esse histórico e rejeita repetições.

Fluxo funcional do RTL

O funcionamento do bloco ocorre em etapas bem definidas, mantendo a coerência entre geração pseudoaleatória e aceite do dado pela interface de saída:

1. **Reset:** o sinal `rst_i` limpa a memória do detector de duplicatas e reinicializa o estado operacional do bloco;
2. **Seleção de seed:** o `rng_selector` escolhe a tabela de permutação ativa a partir de um contador interno;
3. **Geração do candidato:** o `rng_data_path` produz um valor de 3 bits com base na combinação entre seed e índice atual;
4. **Validação:** o `rng_hs_dup_detector` compara o candidato com o histórico armazenado; valores repetidos são descartados e o circuito busca a próxima combinação válida;
5. **Handshake e armazenamento:** quando o consumidor confirma a leitura por `wr_i`, o valor aceito é gravado no histórico de não repetição.

Esse fluxo mostra que a unicidade das saídas não depende apenas da escolha da seed, mas da cooperação entre caminho de dados, lógica de comparação e protocolo de aceite.

Parâmetros e escalabilidade

Embora o sistema não seja totalmente parametrizável devido à limitações nas criações do barramento de memória ainda sim trabalhei da forma mais modular possível:

Parâmetro	Valor	Função no RTL
WIDTH	3	Largura do dado pseudoaleatório, correspondente aos valores de 0 a 7.
T_WIDTH	WIDTH-1	Ajuste auxiliar para declaração de vetores e barramentos.
SEED_TOT_NUMB	12	Quantidade total de números organizados nas sementes e tabelas internas.
COUNT_WIDTH	$\lceil \log_2(\dots) \rceil$	Largura calculada para os contadores associados ao avanço de estados e sequências.

Tabela 3: Parâmetros estruturais descritos na documentação de especificação.

Controle e FSMs

O bloco possui um sistema de máquina de estados complexa que pode ser descrita melhor abaixo:

- **Reset:** Ao iniciar, o sistema deve ser resetado (`rst_i`). Isso limpa a memória do detector de duplicatas, reinicia os contadores e define o estado inicial da FSM.
- **Seleção de Seed:** O módulo `rng_selector` utiliza o clock para incrementar um contador interno. O momento da ativação define qual das 4 tabelas de permutação será usada para a geração atual.
- **Geração e Validação (Loop Interno):** O `rng_data_path` fornece um número candidato baseado na seed e no contador sequencial. O `rng_hs_dup_detector` verifica se este número existe na memória interna (`mem`).
- **Se existir (Duplicata):** O detector ativa o sinal interno `req_new_num_o`. Isso força o sistema a descartar o número atual e buscar o próximo imediatamente na tabela.
- **Se não existir (Único):** O número é disponibilizado em `num_to_send_o`.
- **Confirmação de Leitura:** O usuário/sistema externo deve enviar o sinal `wr_i` para confirmar o uso do número. Isso instrui o detector a armazenar o valor atual na memória de histórico, impedindo que ele seja gerado novamente até que o buffer seja limpo ou reiniciado.

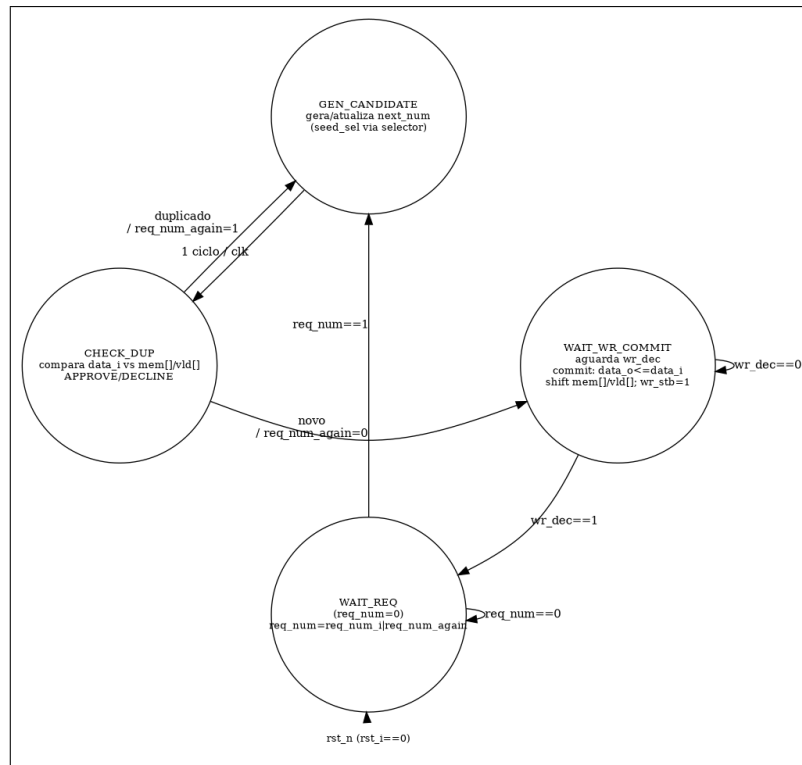


Figura 2: Diagrama de estados da FSM.

Em termos práticos, a sequência de operação pode ser interpretada como um fluxo de estados implícitos:

- **Inicialização**, após reset;
- **Seleção e geração**, quando um novo candidato é produzido;
- **Comparação**, quando o candidato é confrontado com o histórico armazenado;
- **Aceite**, quando `wr_i` confirma a transferência de um valor válido.

Abaixo há uma ilustração pré-projeto do comportamento esperado do circuito:

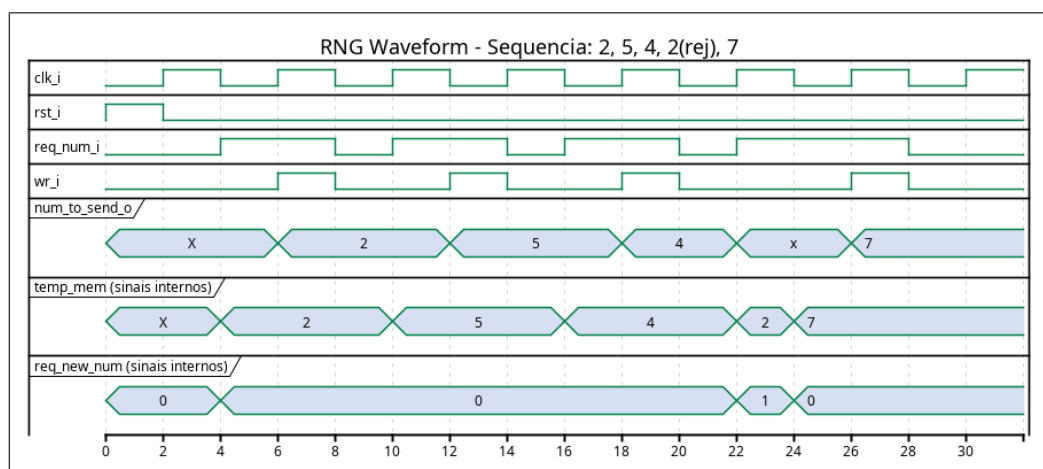


Figura 3: Diagrama de temporização do protocolo de operação do bloco.

3. Estratégia de Verificação e Resultados

A estratégia de verificação do bloco foi estruturada em torno de uma bancada UVM executada com Verilator e analisada com apoio do GTKWave. O foco da campanha foi validar o protocolo de handshaking do gerador pseudo-aleatório, a captura correta das amostras produzidas pelo DUT e a consolidação dos resultados observados no scoreboard.

Visão geral da bancada

- **Bloco verificado:** gerador de números pseudo-aleatórios de 3 bits com não repetição (RNG).
- **Versão da verificação:** 1.0.
- **Responsável pela verificação:** Matheus Grossi.
- **Ferramentas utilizadas:** UVM, Verilator e GTKWave.
- **Período da campanha:** 24/02/2026 a 10/03/2026.

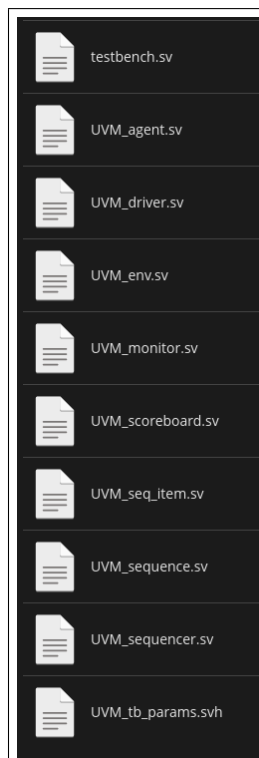


Figura 4: Lista de itens que compõem a bancada.

Objetivos da verificação

O objetivo da verificação foi assegurar os seguintes pontos funcionais e estruturais:

- O protocolo de solicitação e aceite do bloco, por meio dos sinais `req_num_i` e `wr_i`, ocorre conforme o handshaking modelado na bancada;

- O monitor captura corretamente as amostras válidas de `num_to_send_o` na presença de `wr_i`;
- O scoreboard consolida a sequência observada, contabiliza ocorrências e sinaliza repetições reais;
- O DUT é exercitado sob três faixas de temporização configuradas na sequência, com `clk_toggle_tu` igual a 3, 7 e 10, além de espaçamentos pseudoaleatórios entre requisições.

Caso de teste executado

ID do Teste	Descrição	Sequência Principal	Ambiente	Critério de Sucesso
rng_test	Teste principal com 24 rodadas, divididas em três faixas temporais, com intervalos pseudoaleatórios entre requisições. O teste valida o handshaking, a captura das amostras pelo monitor e a consolidação da sequência observada no scoreboard.	rng_sequence	tb_rng, rng_env, rng_agent, rng_driver, rng_monitor e rng_scoreboard	Execução completa das 24 rodadas, ausência de <code>uvm_fatal</code> , publicação de amostras válidas pelo monitor e geração do resumo final do scoreboard com contagem consistente.

Resultados obtidos

- **Total de sequências:** 1 sequência principal, `rng_sequence`.
- **Testes aprovados:** 1 teste estrutural previsto pela bancada.
- **Testes com falha explícita:** 0 falhas estruturais explícitas na bancada.

Do ponto de vista estrutural, os comportamentos esperados foram observados. A bancada verificou corretamente a liberação de reset, a execução das 24 rodadas, a variação temporal do clock por item de sequência, a publicação de amostras pelo monitor quando `wr_i` está ativo e a consolidação das ocorrências no scoreboard. Também foi identificado um comportamento inesperado durante a verificação: uma falha no protocolo de handshaking. Após a execução de lint, os erros no RTL foram localizados e corrigidos, permitindo a continuidade da campanha de verificação com a bancada já estabilizada.

Análise dos resultados

A bancada está organizada em UVM com agente, sequencer, driver, monitor, ambiente e scoreboard conectados de forma coerente. Por se tratar de um projeto residencial e feita em um compilador com restrições de SystemVerilog não foi feita uma cobertura mais aprofundada, para um caso de projeto comercial desse bloco recomenda-se uma abordagem mais densa de verificação.

Cobertura de verificação

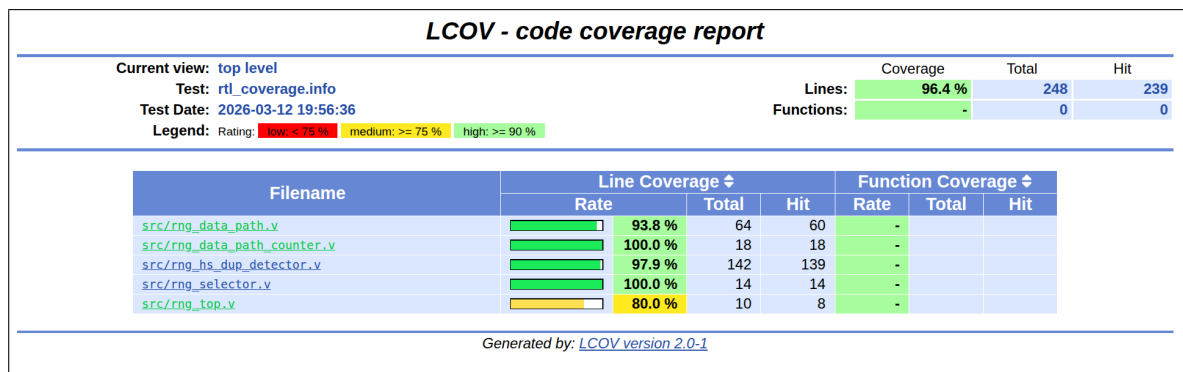


Figura 5: Lista do relatório de cobertura da bancada.

A cobertura atualmente obtida é predominantemente **estrutural e observacional**. O monitor coleta amostras válidas da saída e o scoreboard resume histograma, quantidade de valores únicos, duplicatas reais e sequência observada. Entretanto, ainda não há métricas formais de cobertura para:

- todos os valores possíveis de 3 bits;
- a exaustão completa do conjunto sem repetição;
- as transições entre estados internos do DUT;
- cenários de erro ou de violação de protocolo.

Limitações observadas

Foi registrado que o desenvolvimento da bancada ainda pode evoluir em profundidade metodológica. Em particular, a ausência de um modelo de referência, de cobertura funcional e de checagens mandatórias para cenários negativos indica espaço para amadurecimento da estratégia de verificação nas próximas iterações.

Conclusão da verificação

A bancada UVM do bloco RNG foi concluída com sucesso e mostrou-se apta para uso em simulação e depuração do DUT. Como próximos avanços naturais, a campanha pode incorporar cobertura funcional, um modelo de referência explícito e critérios de falha mais rigorosos para o requisito de não repetição.

4. Síntese Lógica e Resultados PPA

Neste projeto foram adotados dois cenários: uma síntese direcionada a FPGA para inspeção estrutural do hardware e um fluxo ASIC baseado no ambiente *Librelane*, com mapeamento tecnológico para a biblioteca de *standard cells* *sky130_fd_sc_hd* do PDK SkyWater 130 nm. Como o projeto foi tratado como prova de conceito e não como candidato imediato a *tape-out*, as restrições temporais aplicadas foram genéricas, de modo que os resultados de performance e potência devem ser interpretados como preliminares e dependentes de refinamento posterior.

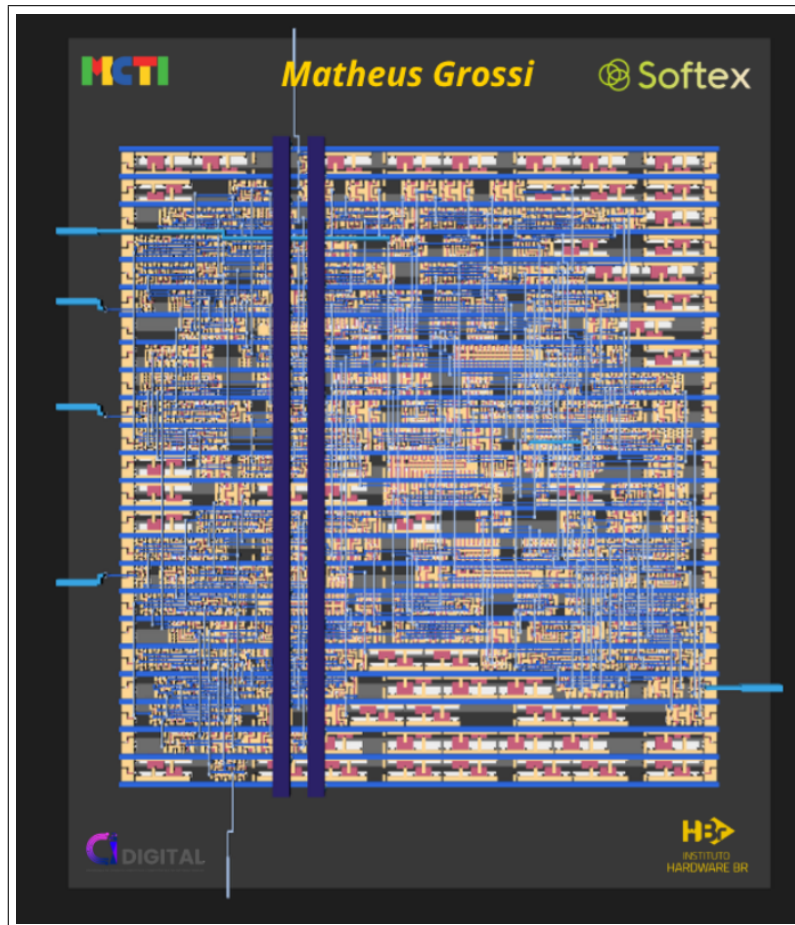


Figura 6: Vista pós-síntese do SoC apresentada na documentação-base.

Considerações de ASIC

Esse bloco possui 3 pontos importantes a mencionar:

- **Domínio de clock único:** o bloco opera a partir do sinal `clk_i`, sem múltiplos domínios de clock;
- **Reset bem definido:** o sinal `rst_i` é assíncrono e ativo em nível baixo, permitindo limpeza imediata do histórico e reinicialização do fluxo operacional;
- **Estilo sintetizável:** a separação entre blocos de dados, seleção e armazenamento favorece uma descrição RTL compatível com síntese e reduz o risco de *latches*

indesejados, desde que a codificação combinacional mantenha atribuições completas.

Além disso, a existência de um contador, tabelas de permutação e memória de histórico torna o projeto naturalmente adequado a estudos comparativos entre FPGA e ASIC, especialmente em área, temporização e custo de implementação do mecanismo de não repetição.

A síntese baseada em FPGA foi feita com base nas constraints genéricas do Basys2 e os resultados obtidos podem ser vistos abaixo:

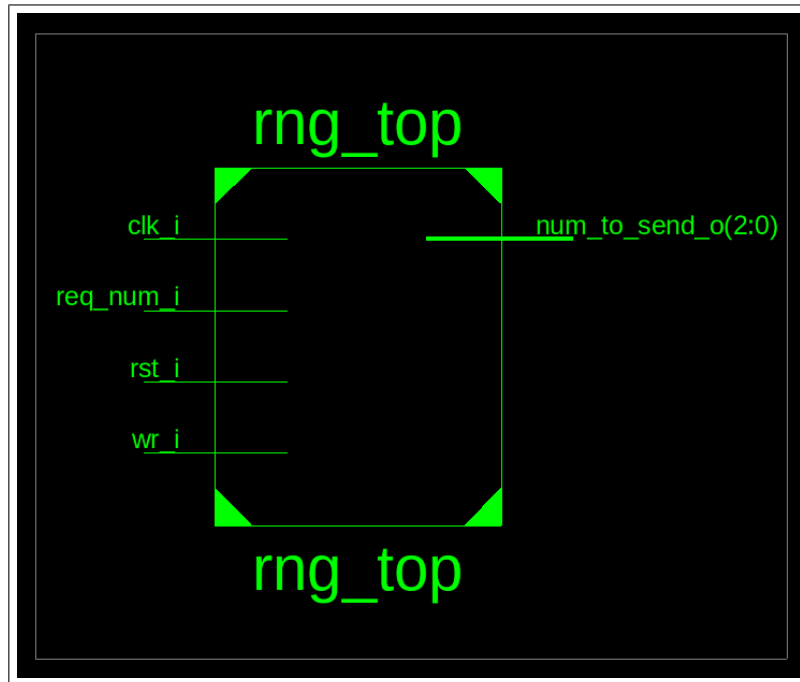


Figura 7: Visão de Topo do Circuito (Síntese)

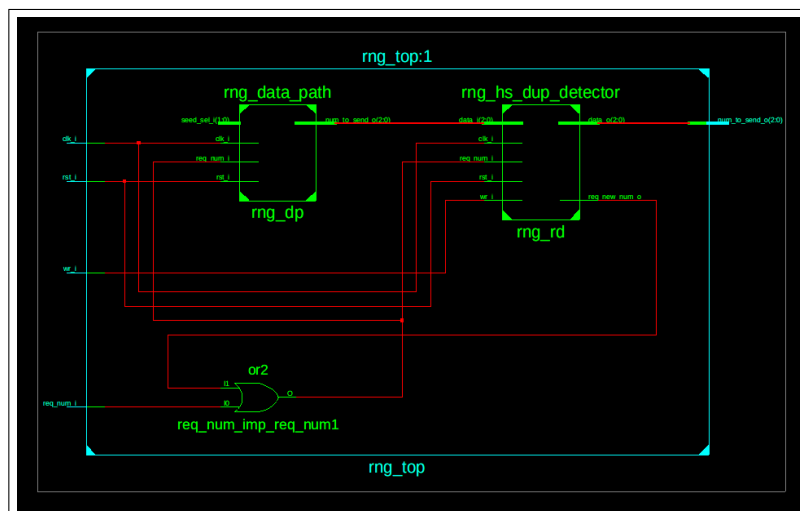


Figura 8: Vista Explodida 1

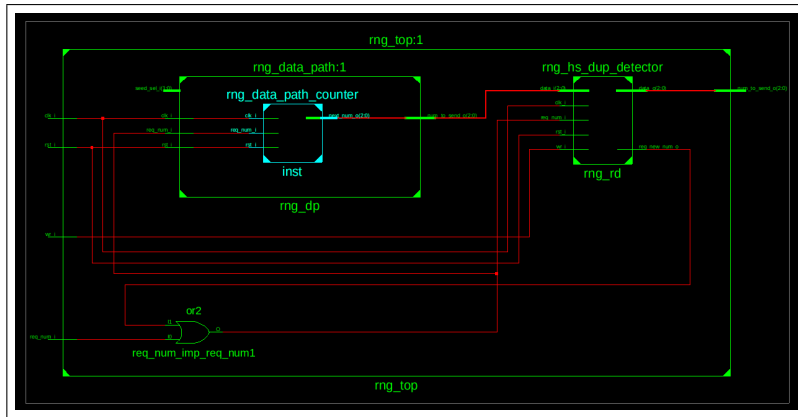


Figura 9: Vista Explodida 2

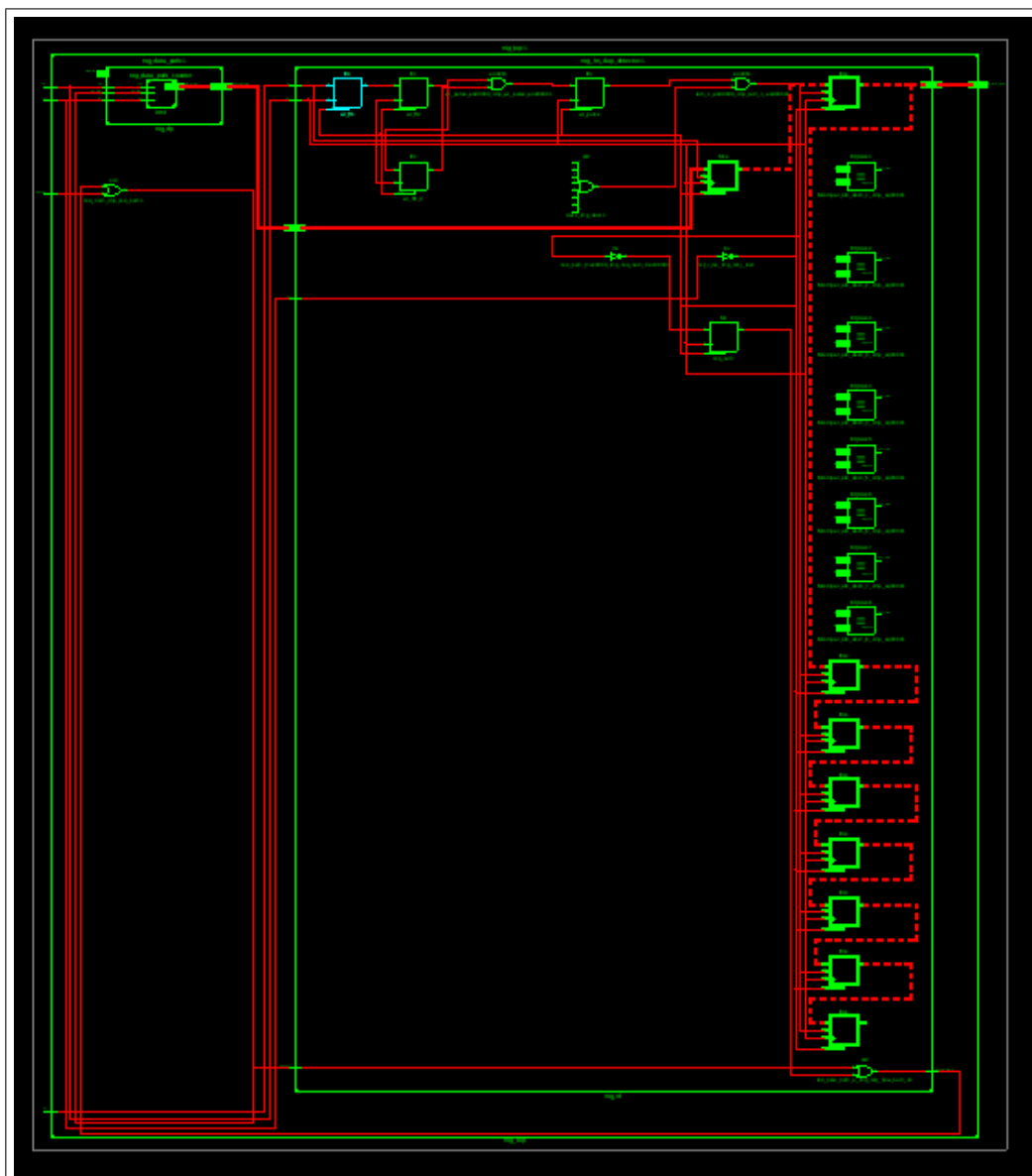


Figura 10: Vista Explodida 3

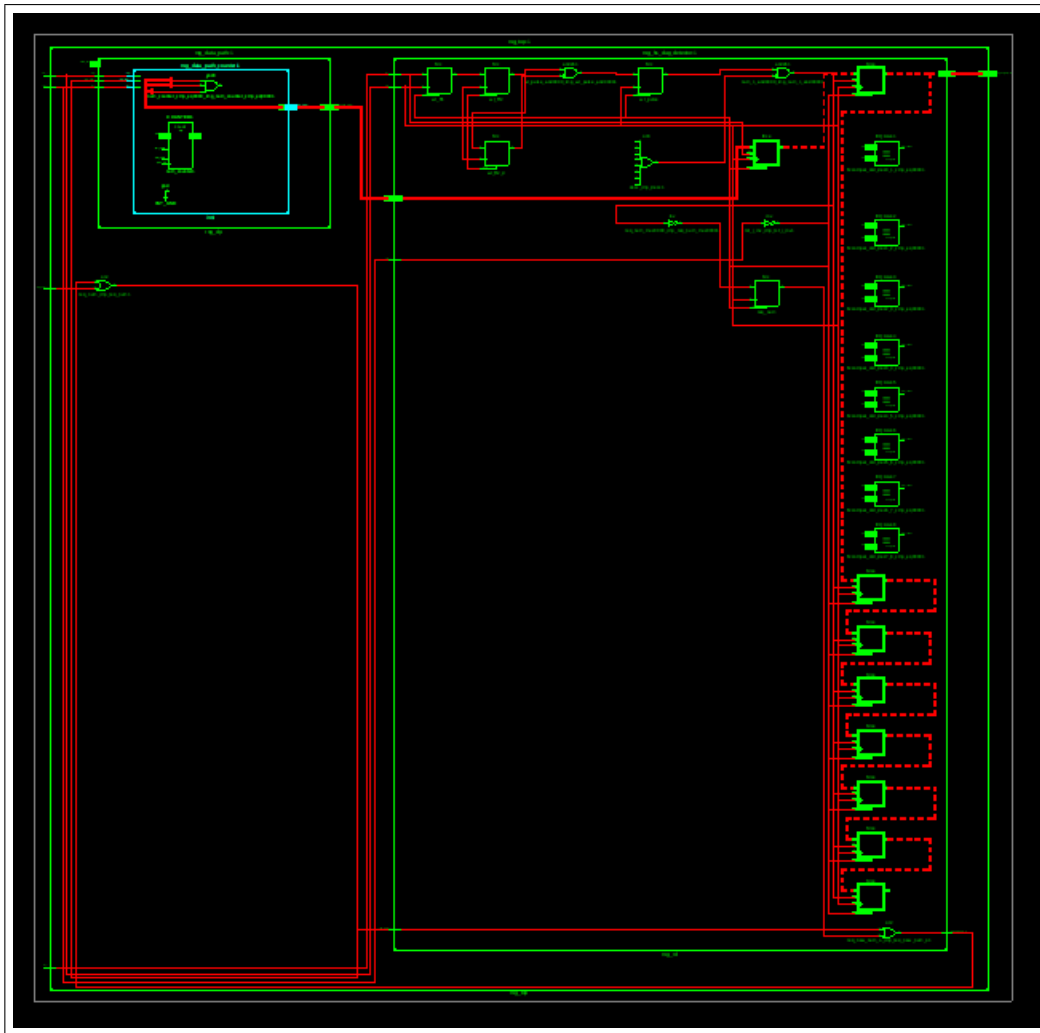


Figura 11: Vista Explodida 4

Fluxo de síntese adotado

No fluxo ASIC descrito no material-base, a preparação do projeto parte da criação de uma pasta dedicada dentro de `openframe_user/openlane`, contendo um subdiretório `src` para os arquivos RTL e um arquivo `config.json` na raiz do projeto. Esse arquivo centraliza os parâmetros básicos de síntese e roteamento antes da aplicação das *constraints* SDC. O documento também explicita que os ficheiros `.v` e `.sv` do projeto foram concentrados na pasta `src`, mantendo uma organização compatível com o fluxo padrão da ferramenta.

```
matheus@matheus-Vivobook-Go-E1504FA-E1504FA:~/openframe_user_project/openlane/rng$ ls -la
total 20
drwxr-xr-x  4 matheus matheus 4096 mar 12 20:01 .
drwxrwxr-x 11 matheus docker 4096 fev 17 10:42 ..
-rw-rw-r--  1 matheus matheus  439 mar  9 20:42 config.json
drwxr-xr-x  8 matheus matheus 4096 mar 12 20:08 runs
drwxr-xr-x  3 matheus matheus 4096 mar 12 23:00 src
```

Figura 12: Organização da árvore de projeto.

```

{
  "DESIGN_NAME": "rng_top",
  "VERILOG_FILES": "dir::src/*.v",
  "CLOCK_PORT": "clk_i",
  "CLOCK_PERIOD": 3125,
  "RSZ_DONT_TOUCH": ["*_DONOTTOUCH"],
  "FP_SIZING": "absolute",
  "DIE_AREA": "0 0 70 85",
  "FP_CORE_UTIL": "50",
  "FP_ASPECT_RATIO": 1.0,
  "FP_CORE_MARGIN": 2,
  "PL_TARGET_DENSITY": 0.6,
  "FP_PDN_MULTILAYER": true,
  "SYNTH_ABC_LEGACY_REFACTOR": false,
  "SYNTH_ABC_LEGACY_REWRITE": 0
}

```

Figura 13: Configuração do arquivo `config.json` utilizado na síntese.

Restrições e premissas de implementação

As restrições de projeto foram mantidas em um nível genérico, sem um arquivo SDC ajustado especificamente para metas agressivas de frequência, fechamento temporal fino ou otimização direcionada a um *sign-off* de produção. Essa escolha é coerente com o caráter acadêmico e exploratório do trabalho, mas impõe uma limitação importante à leitura de PPA: quaisquer conclusões quantitativas sobre frequência máxima, *slack*, área física final e potência devem ser tratadas como indicativas, e não como números finais de projeto.

Resultados da síntese lógica

A tradução do módulo `rng_top` para nível de portas preservou a decomposição funcional apresentada nas seções anteriores, reunindo caminho de dados, seletor de sementes, detector de duplicatas e contador cíclico. O mapeamento lógico resultou em uso intensivo de multiplexadores para roteamento de dados e de portas XOR para as etapas de validação e contagem. O dado quantitativo explicitamente informado é a utilização total de **122 células lógicas**, valor que caracteriza uma implementação compacta para o porte do bloco e coerente com a simplicidade relativa do algoritmo e da interface de handshaking.

```

=== rng_top ===

Number of wires:          106
Number of wire bits:      126
Number of public wires:   21
Number of public wire bits: 41
Number of ports:          5
Number of port bits:      7
Number of memories:       0
Number of memory bits:    0
Number of processes:      0
Number of cells:          122
  $_ANDNOT_               19
  $_AND_                   1
  $_MUX_                   30
  $_NAND_                  1
  $_ORNOT_                 1
  $_OR_                    9
  $_XNOR_                  8
  $_XOR_                   18
  sky130_fd_sc_hd__dfntp_2 35

Chip area for module 'rng_top': 919.632000
of which used for sequential elements: 919.632000 (100.00%)

```

Figura 14: Resumo da composição lógica do circuito apresentado na documentação de síntese.

Leitura dos resultados de PPA

Embora a seção original faça referência a PPA, o material fornecido não traz todos os números clássicos de *Performance, Power and Area* já consolidados em formato tabular. Em particular, a documentação não explicita frequência máxima atingida, *Worst Negative Slack* (WNS), área total em μm^2 ou equivalência em portas NAND2. Ainda assim, ela permite registrar de forma objetiva o estado atual da caracterização do bloco:

No caso específico da potência, o texto destaca corretamente que os relatórios estáticos de síntese (como `stat.rpt` e `stat.json`) não bastam para uma estimativa dinâmica realista. Para obter *dynamic power* e *leakage* com maior fidelidade, o fluxo precisa incorporar dados de atividade de comutação oriundos das simulações pós-síntese, normalmente por meio de ficheiros `.vcd` gerados na pasta de *waveforms* e consumidos por uma ferramenta de *Power Analysis*. Assim, a caracterização energética do bloco ainda deve ser vista como pendente de fechamento metodológico.

```

=====
report_power
=====
===== nom_tt_025C_1v80 Corner =====

```

Group	Internal Power	Switching Power	Leakage Power	Total Power (Watts)
Sequential	6.595862e-07	1.464358e-07	4.117267e-10	8.064338e-07 33.7%
Combinational	6.884237e-07	4.226346e-07	5.090588e-10	1.111567e-06 46.5%
Clock	3.096065e-07	1.619649e-07	2.217573e-10	4.717931e-07 19.7%
Macro	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00 0.0%
Pad	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00 0.0%
Total	1.657616e-06	7.310353e-07	1.142543e-09	2.389794e-06 100.0%
	69.4%	30.6%	0.0%	

Figura 15: Resumo visual das métricas de potência referenciadas no material-base.

Discussão técnica

Do ponto de vista de área e complexidade lógica, o resultado de 122 células sugere que o bloco possui custo de implementação modesto, o que é compatível com uma arquitetura síncrona de propósito específico e com largura de dados reduzida. Em contrapartida, a ausência de *constraints* refinadas e de números temporais consolidados impede afirmar, nesta fase, qual é o limite real de desempenho do circuito em ASIC. Em outras palavras, a síntese lógica já demonstra viabilidade de implementação, mas a etapa de PPA ainda precisa ser aprofundada com relatórios temporais completos, extração pós-roteamento e análise de potência baseada em atividade.

5. Conclusão e Lições Aprendidas

- Muitas dificuldades surgiram ao longo do projeto, até que a versão final fosse alcançada passamos por 7 revisões na arquitetura do sistema de handshaking.
- Enquanto fazia a verificação descobri um laço ciclico que prendia a operação do circuito e isso só foi descoberto após um lint severo na bancada.
- Pretendo fazer melhorias na versão atual para expandir o sistema e adequa-lo em um outro projeto que farei.
- Pretendo portar o projeto para um fluxo em systemverilog no RTL o que me permitirá deixa-lo 100% parametrizável.

6. Referências e Apêndices

- Link para o repositório Git.
- Scripts de síntese (TCL/SDC) simplificados.
- Referências bibliográficas e manuais de ferramentas utilizados.